



Trabalhando com imagens em Python

Caio Mesquita

Objetivos da atividade



- Ao final da atividade, o aluno será capaz de:
 - Entender como uma imagem é representada
 - Interpretar imagens como matrizes numéricas
 - Manipular imagens usando Python
 - Aplicar transformações como:
 - Inversão de cores
 - Rotação
 - Alongamento (escala)
 - Espelhamento
 - Aplicação de aprendizado de máquina em imagens digitais

O que é uma imagem digital?



- Uma imagem digital (em preto e branco) é uma **matriz de pixels**
- Cada pixel possui um valor numérico (entre 0 e 255)
- Em imagens preto e branco
 - 0 -> preto
 - 255 -> branco
 - Valores entre 0 e 255 -> tons de cinza
- Operações matriciais correspondem a mudanças na imagem

O que é uma imagem digital?



[8]

✓ Os

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

[5]

✓ Os

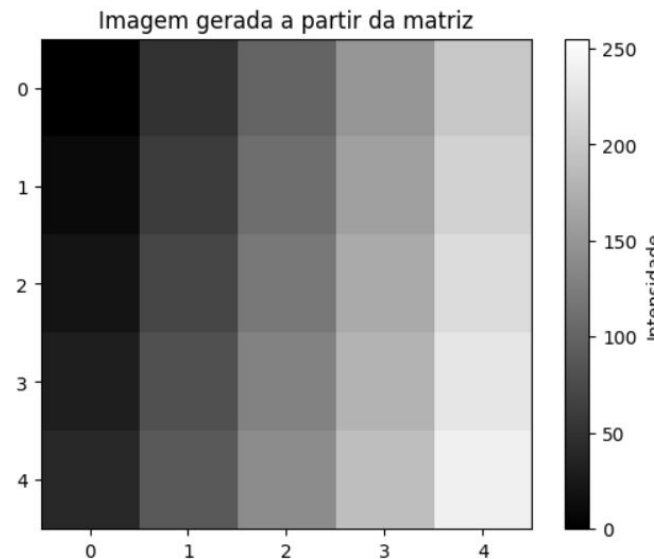
```
▶ I = np.array([
    [ 0, 50, 100, 150, 200],
    [10, 60, 110, 160, 210],
    [20, 70, 120, 170, 220],
    [30, 80, 130, 180, 230],
    [40, 90, 140, 190, 240]
])
```

```
print("Matriz I:")
print(I)
```

▼

```
... Matriz I:
[[ 0 50 100 150 200]
 [10 60 110 160 210]
 [20 70 120 170 220]
 [30 80 130 180 230]
 [40 90 140 190 240]]
```

```
plt.imshow(I, cmap='gray', vmin=0, vmax=255)
plt.title("Imagem gerada a partir da matriz")
plt.colorbar(label="Intensidade")
plt.show()
```



Carregando uma imagem



[15]

✓ 3s



```
pip install scikit-image
```

[16]

✓ 0s

```
from skimage import data

# Carrega imagem de exemplo (já vem na biblioteca)
img = data.camera() # imagem clássica em escala de cinza

print("Dimensões:", img.shape)
print("Tipo:", img.dtype)

plt.imshow(img, cmap='gray', vmin=0, vmax=255)
plt.title("Imagem da biblioteca (camera)")
plt.axis("off")
plt.show()
```

... Dimensões: (512, 512)
Tipo: uint8

Imagem da biblioteca (camera)



Manipulando a imagem como matriz



```
print(img)
```

```
[[200 200 200 ... 189 190 190]
 [200 199 199 ... 190 190 190]
 [199 199 199 ... 190 190 190]
 ...
 [ 25  25  27 ... 139 122 147]
 [ 25  25  26 ... 158 141 168]
 [ 25  25  27 ... 151 152 149]]
```

```
▶ print(img[:5, :5])
```

```
... [[200 200 200 200 199]
      [200 199 199 200 199]
      [199 199 199 200 200]
      [200 200 199 199 199]
      [200 200 200 200 199]]
```

Inversão de cores



```
imagem_invertida = 255 - img
```

```
print("Dimensões:", imagem_invertida.shape)  
print("Tipo:", imagem_invertida.dtype)
```

```
plt.imshow(imagem_invertida, cmap='gray', vmin=0, vmax=255)  
plt.title("Imagem da biblioteca (camera)")  
plt.axis("off")  
plt.show()
```

```
print(imagem_invertida)
```

```
[[ 55  55  55 ...  66  65  65]  
 [ 55  56  56 ...  65  65  65]  
 [ 56  56  56 ...  65  65  65]  
 ...  
 [230 230 228 ... 116 133 108]  
 [230 230 229 ...  97 114  87]  
 [230 230 228 ... 104 103 106]]
```

Dimensões: (512, 512)

Tipo: uint8

Imagem da biblioteca (camera)



Rotação da imagem



```
img_rotacionada = np.rot90(img)

plt.imshow(img_rotacionada, cmap='gray')
plt.title("Imagem Rotacionada 90°")
plt.axis("off")
plt.show()
```

Imagem Rotacionada 90°



Espelhamento horizontal



```
img_espelhada = np.fliplr(img)

plt.imshow(img_espelhada, cmap='gray')
plt.title("Espelhamento Horizontal")
plt.axis("off")
plt.show()
```

Espelhamento Horizontal



Alongamento Horizontal



```
img_redimensionada = np.repeat(img, 2, axis=1)

print("Novas dimensões:", img_redimensionada.shape)
plt.imshow(img_redimensionada, cmap='gray')
plt.title("Alongamento Horizontal (NumPy)")
plt.axis("off")
plt.show()
```

Novas dimensões: (512, 1024)

Alongamento Horizontal (NumPy)



Aumentar brilho



```
img_clara = np.clip(img + 50, 0, 255)
#np.clip() evita valores fora do intervalo 0-255
plt.imshow(img_clara, cmap='gray')
plt.title("Brilho Aumentado")
plt.axis("off")
plt.show()
```

Brilho Aumentado



Exercício

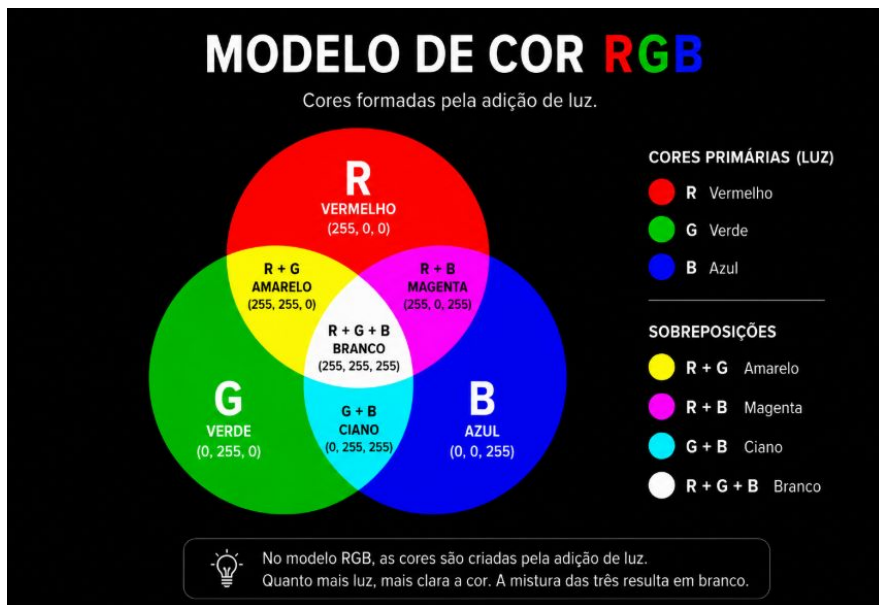


- Carregar uma imagem em preto e branco:
`Img = data.page()`
- Inverter as cores
- Rotacionar 180°
- Criar versão espelhada
- Aumentar o brilho

Modelo RGB



- RGB(Red, Green, Blue) é um modelo aditivo de cores
- As cores podem ser combinadas em diferentes intensidades para formar novas cores



Representação digital



- No computador cada pixel é um vetor: [R, G, B]
- [255, 0, 0] = **vermelho puro**
- [0, 255, 0] = **verde puro**
- [0, 0, 255] = **azul puro**
- [255, 255, 255] = **branco**
- [0, 0, 0] = **preto**

Matriz de vetores



```
import numpy as np
```

```
imagem = np.array([
    [[255, 0, 0], [0, 255, 0], [0, 0, 255], [255, 255, 0]],
    [[255, 0, 255], [0, 255, 255], [128, 128, 128], [255, 255, 255]],
    [[0, 0, 0], [50, 100, 150], [200, 50, 100], [25, 75, 125]],
    [[10, 20, 30], [60, 70, 80], [90, 100, 110], [120, 130, 140]]
])
```

```
print(imagem)
```

```
[[[255  0  0]
   [ 0 255  0]
   [ 0  0 255]
   [255 255  0]]
```

```
[[255  0 255]
 [ 0 255 255]
 [128 128 128]
 [255 255 255]]
```

```
[[ 0  0  0]
 [ 50 100 150]
 [200  50 100]
 [ 25  75 125]]
```

```
[[ 10  20  30]
 [ 60  70  80]
 [ 90 100 110]
 [120 130 140]]]
```

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

```
plt.imshow(imagem)
plt.axis("off")
plt.show()
```

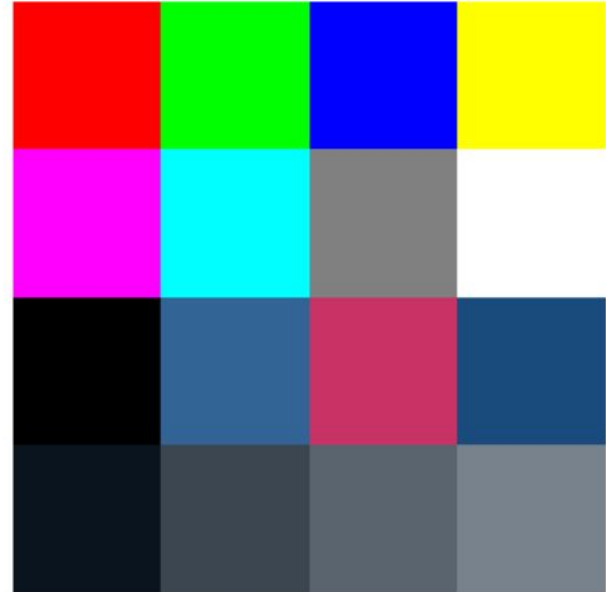


Imagem colorida em Python



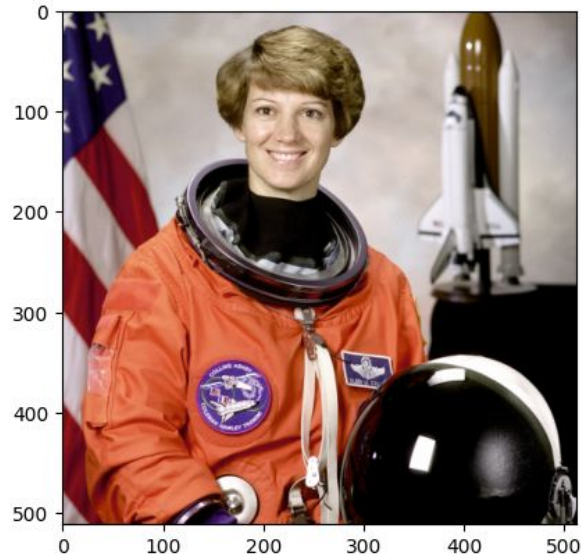
```
from skimage import data
import matplotlib.pyplot as plt

# imagem colorida pronta da biblioteca
img = data.astronaut()

print(img.shape)

plt.imshow(img)
```

(512, 512, 3)
<matplotlib.image.AxesImage at 0x7b63043d59d0>



Manipulando as imagens em Python



```
imagem_invertida = 255 - img
```

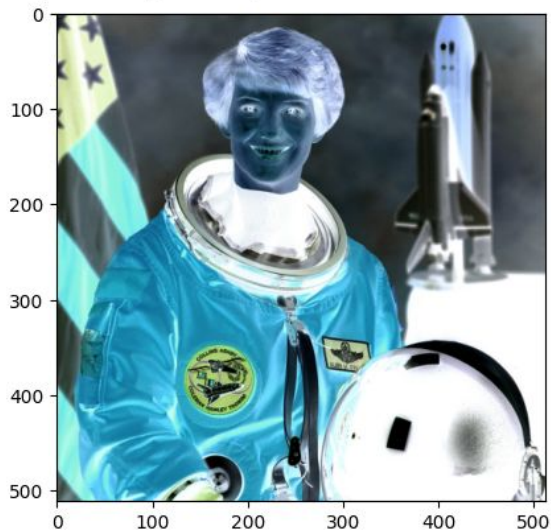
```
print("Dimensões:", imagem_invertida.shape)  
print("Tipo:", imagem_invertida.dtype)
```

```
plt.imshow(imagem_invertida)
```

Dimensões: (512, 512, 3)

Tipo: uint8

<matplotlib.image.AxesImage at 0x7b62fb0525a0>



```
img_rotacionada = np.rot90(img)
```

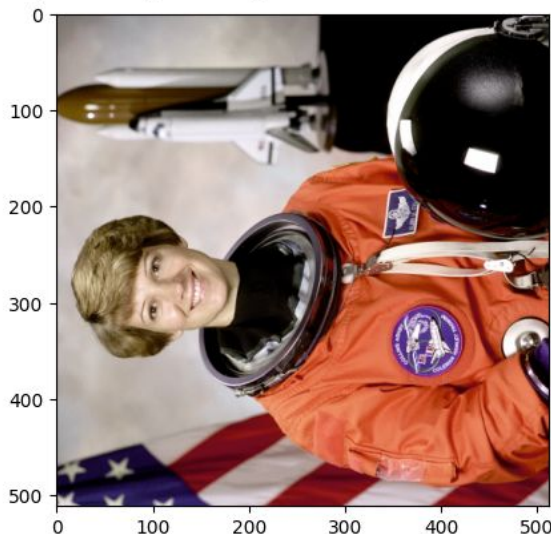
```
print("Dimensões:", img_rotacionada.shape)  
print("Tipo:", img_rotacionada.dtype)
```

```
plt.imshow(img_rotacionada)
```

Dimensões: (512, 512, 3)

Tipo: uint8

<matplotlib.image.AxesImage at 0x7b62fb091dc0>



```
img_espelhada = np.fliplr(img)
```

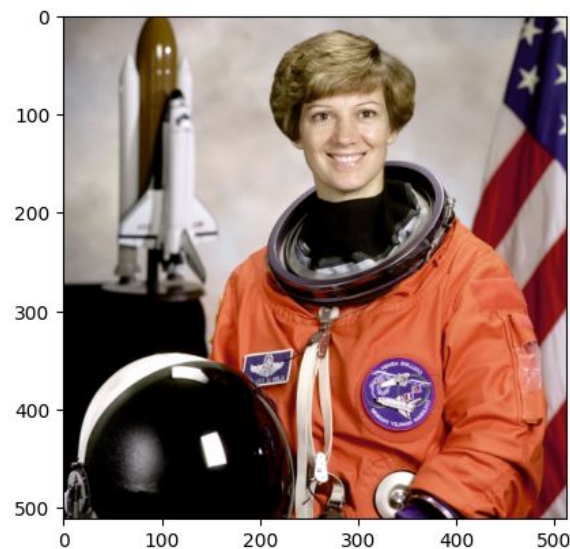
```
print("Dimensões:", img_espelhada.shape)  
print("Tipo:", img_espelhada.dtype)
```

```
plt.imshow(img_espelhada)
```

Dimensões: (512, 512, 3)

Tipo: uint8

<matplotlib.image.AxesImage at 0x7b62fb091a90>



Tratamento de Imagens para Classificação Binária



- Problema
 - Temos imagens em preto e branco e queremos utilizá-las em um modelo de classificação binária
- Uma imagem em escala cinza é representada como uma matriz de pixels
 - Por exemplo:

$$\begin{bmatrix} 0 & 255 & 128 \\ 64 & 192 & 255 \\ 0 & 128 & 64 \end{bmatrix}$$



- Conversão para vetor:
 - Algoritmos de aprendizado de máquina esperam que cada amostra seja um vetor de características
 - A matriz da imagem é transformada em um vetor unidimensional (flatten)

$$\begin{bmatrix} 0 & 255 & 128 \\ 64 & 192 & 255 \\ 0 & 128 & 64 \end{bmatrix} \rightarrow [0, 255, 128, 64, 192, 255, 0, 128, 64]$$

Tratamento de Imagens para Classificação Binária



Amostra (Imagem)	Pixel 1	Pixel 2	Pixel 3	Pixel 4	...	Pixel n	Classe
Img 1	0	255	128	64	...	32	0
Img 2	255	255	0	128	...	64	1
Img 3	128	64	255	192	...	0	0

$$X = \begin{bmatrix} 0 & 255 & 128 & 64 & \dots & 32 \\ 255 & 255 & 0 & 128 & \dots & 64 \\ 128 & 64 & 255 & 192 & \dots & 0 \end{bmatrix}$$

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

- Cada linha de X corresponde a uma imagem
- Cada coluna de X corresponde a um pixel específico
- Y contem os rótulos da classificação binária

Vetorizando a matriz em Python (imagem cinza)



```
import numpy as np

matriz = np.array([
    [0, 255, 128],
    [64, 192, 255],
    [0, 128, 64]
])
```

```
print(matriz)
```

```
[[ 0 255 128]
 [ 64 192 255]
 [ 0 128 64]]
```

```
vetor = matriz.flatten()
```

```
print(vetor)
```

```
[ 0 255 128 64 192 255 0 128 64]
```

Vetorizando a matriz em Python (imagem colorida)



```
import numpy as np
```

```
imagem = np.array([
    [[255, 0, 0], [0, 255, 0], [0, 0, 255], [255, 255, 0]],
    [[255, 0, 255], [0, 255, 255], [128, 128, 128], [255, 255, 255]],
    [[0, 0, 0], [50, 100, 150], [200, 50, 100], [25, 75, 125]],
    [[10, 20, 30], [60, 70, 80], [90, 100, 110], [120, 130, 140]]
])
```

```
print(imagem)
```

```
[[[255  0  0]
   [ 0 255  0]
   [ 0  0 255]
   [255 255  0]]
```

```
[[[255  0 255]
   [ 0 255 255]
   [128 128 128]
   [255 255 255]]
```

```
[[[ 0  0  0]
   [ 50 100 150]
   [200 50 100]
   [ 25 75 125]]
```

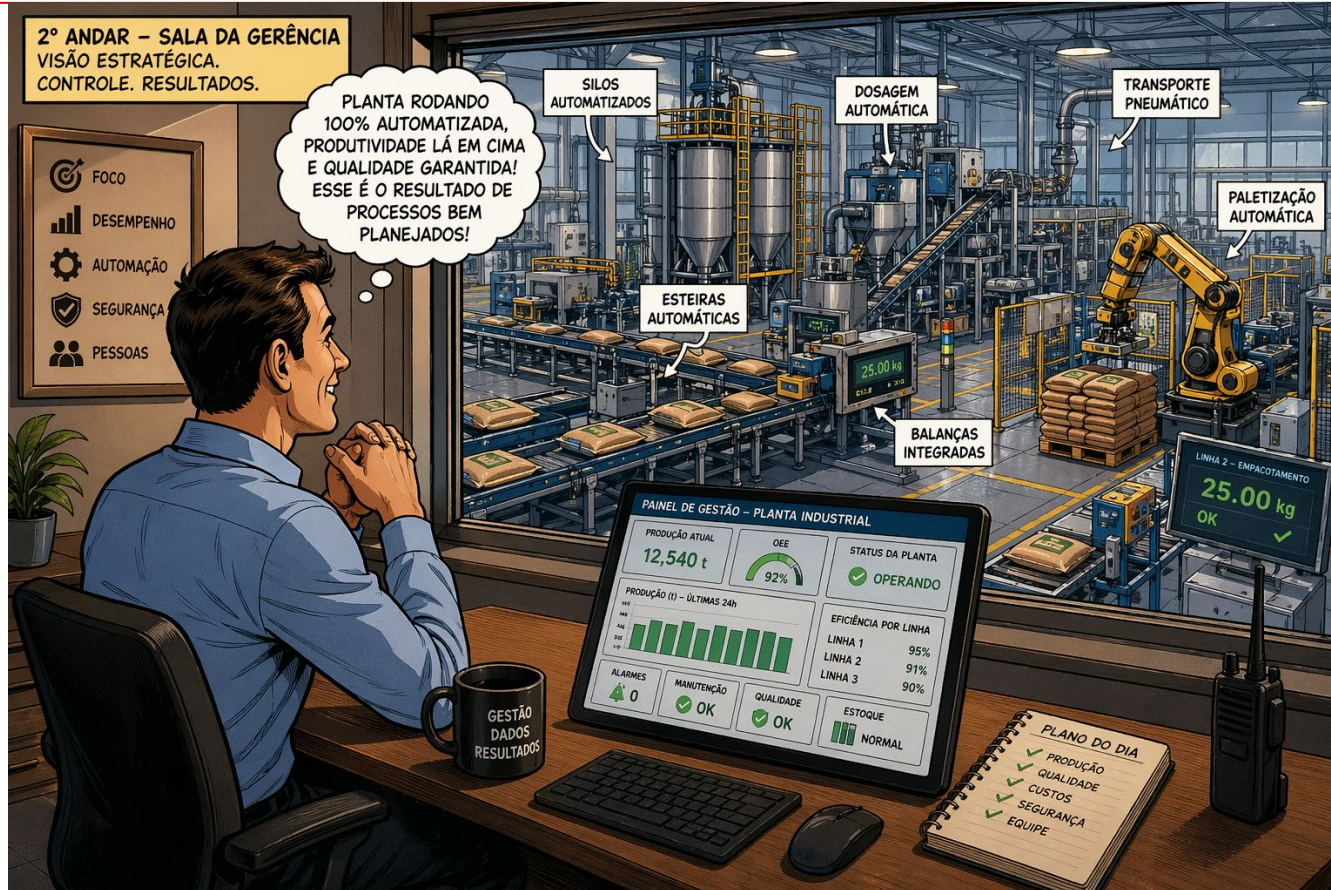
```
[[[ 10 20 30]
   [ 60 70 80]
   [ 90 100 110]
   [120 130 140]]]
```

```
vetor = imagem.flatten()
```

```
print(vetor)
```

```
[255  0  0  0 255  0  0  0 255 255 255  0 255  0 255  0 255 255
128 128 128 255 255 255  0  0  0  50 100 150 200  50 100  25  75 125
 10  20  30  60  70  80  90 100 110 120 130 140]
```


Imagine a seguinte situação



Imagine a seguinte situação



Imagine a seguinte situação



DOIS CENÁRIOS. AMBOS CATASTRÓFICOS. QUAL DECISÃO TOMAR?

CENÁRIO 1: DEIXAR TUDO PARADO

BALANÇA OS
FALHA
DETECTADA

PRODUÇÃO PARADA

0.00 kg
ERRO

- ✓ AGUARDAR TÉCNICO PARA CONSERTAR A BALANÇA
- ✓ PRODUÇÃO TOTALMENTE PARADA O DIA INTEIRO
- ✓ METAS NÃO CUMPRIDAS
- ✓ PREJUÍZOS ELEVADOS

✗ O DIRETOR NÃO VAI GOSTAR NEM UM POUCO!

CENÁRIO 2: OPERAR SEM BALANÇA

BALANÇA OS
FALHA
DETECTADA

OPERANDO SEM BALANÇA

SOBRECARGA NO MOTOR
127%

- ✓ PRODUÇÃO CONTINUA (POR ENQUANTO)
- ✓ RISCO DE MOTOR SOBRECARREGAR
- ✓ SE O MOTOR QUEBRAR:
 - ALÉM DA BALANÇA, TEREMOS QUE CONSERTAR O MOTOR
 - MAIS DIAS DE PRODUÇÃO PARADA

✗ O DIRETOR VAI ODIAR!

AS DUAS ESCOLHAS PODEM SER DESASTROSAS! QUALQUER DECISÃO PODE GERAR PREJUÍZOS GIGANTES E A IRA DO DIRETOR!

PAINEL DE GESTÃO - PLANTA INDUSTRIAL

PRODUÇÃO ATUAL: 0 t

OEE: 0%

STATUS DA PLANTA: PARADA

PRODUÇÃO (t) - ÚLTIMAS 24h

EFICIÊNCIA POR LINHA

LINHA 1	0%
LINHA 2	0%
LINHA 3	0%

ALARMES: 1

MANUTENÇÃO: URGENTE

QUALIDADE: OK

ESTOQUE: NORMAL

PRODUÇÃO PARADA O DIA INTEIRO? ISSO É INADMISSÍVEL!

VOCÊ COLOCOU O MOTOR EM RISCO? AGORA QUEBROU? VAI CUSTAR CARO E VAMOS FICAR AINDA MAIS DIAS PARADOS! EU ODEIO SURPRESAS!

Imagine a seguinte situação



ESPERA... E SE EXISTIR UMA FORMA INTELIGENTE DE CONTINUAR PRODUZINDO SEM COLOCAR O MOTOR EM RISCO?

TEMOS CAMERAS COM VISÃO TÉRMICA NA PLANTA! COMBINANDO CIÊNCIAS TÉRMICAS, INSTRUMENTAÇÃO E INTELIGÊNCIA ARTIFICIAL, POSSO CRIAR UMA SOLUÇÃO QUE PODE SER A **SALVAÇÃO!**

A IDEIA BRILHANTE!

USAR VISÃO TÉRMICA PARA MONITORAR O MOTOR E IDENTIFICAR SE ESTÁ EM CARGA NORMAL OU SOBRECARGA.

CARGA NORMAL 120 °C
20

SOBRECARGA 120 °C
20

VIÃO TÉRMICA ✓
CIÊNCIAS TÉRMICAS ✓
INSTRUMENTAÇÃO ✓
INTELIGÊNCIA ARTIFICIAL ✓

ESTEIRA PARADA
0.00 kg
ERRO

DASHBOARD DE MONITORAMENTO INTELIGENTE

ESTADO ATUAL DO MOTOR: **CARGA NORMAL**
TEMPO ACUMULADO: **CARGA NORMAL 02:17:45**
SOBRECARGA 00:03:12
LIMITE DE SOBRECARGA 00:05:00
TEMPERATURA ATUAL **78 °C**
64%
SISTEMA PROTETIVO ATIVO

IMAGEM TÉRMICA → ALGORITMO (IA / MACHINE LEARNING) → CLASSIFICAÇÃO
CARGA NORMAL
SOBRECARGA

ASSIM, POSSO CONTROLAR QUANTO TEMPO O MOTOR OPERA EM CADA ESTADO, EVITANDO SOBRECARGAS PROLONGADAS E PROTEGENDO O EQUIPAMENTO.

COMO VAI FUNCIONAR:

1. CÂMERA TÉRMICA MONITORA O MOTOR
2. IA ANALISA AS IMAGENS EM TEMPO REAL
3. CLASSIFICA O ESTADO DO MOTOR
CARGA NORMAL
SOBRECARGA
4. CONTROLA O TEMPO EM SOBRECARGA
5. PROTEGE O MOTOR E MANTÉM A PRODUÇÃO

VOU APRESENTAR ISSO AO DIRETOR! VAMOS TRANSFORMAR UM PROBLEMA EM UMA GRANDE VANTAGEM COMPETITIVA!

Desafio

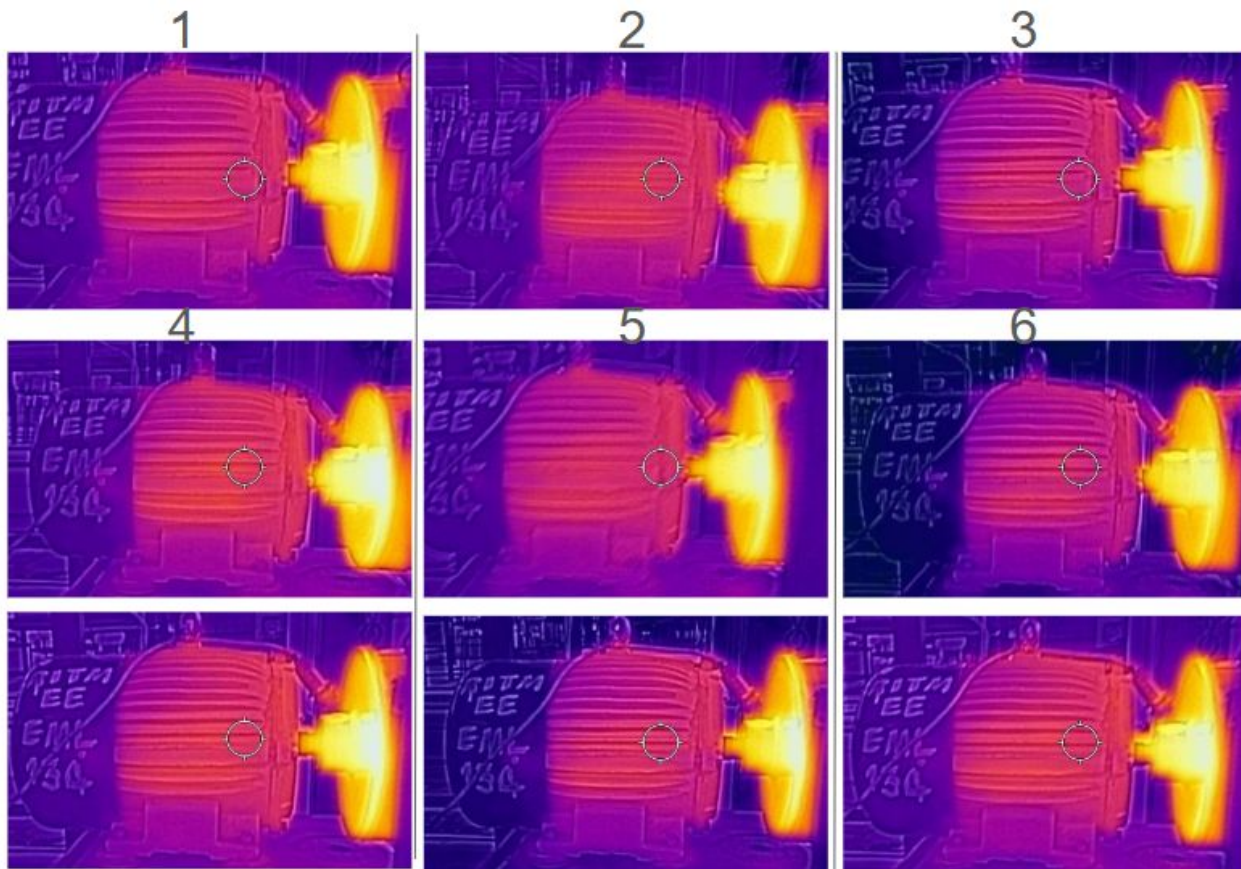


- Dada as imagens a seguir, tente identificar visualmente quais representam o motor operando em carga máxima e quais representam o motor operando em carga normal.

Imagem 0



Imagens



Desafio



- Quais das 10 imagens apresentadas, representam o motor com carga máxima?
- Quais das 10 imagens apresentadas, representam o motor com carga normal?

Exercício



1. Implementar os seguintes algoritmos (usando biblioteca scikit-learn) :
 - Knn
 - Árvore de decisão
 - Random Forest
 - SVM com kernel linear
 - SVM com kernel polinomial de 2° grau
2. Para cada algoritmo, ajustar os principais hiperparâmetros
3. Com os algoritmos treinados, calcular a matriz de confusão para o conjunto de teste
4. Implementar um ensemble com os algoritmos (voto da maioria). Analisar os resultados via matriz de confusão

Exercício (Extra - 1)



- A partir da base de dados `emnist_filtrado_A1.npz` :
 - Aplicar o algoritmo KNN para classificação binária
 - sem uso de bibliotecas (ex: scikit-learn)
 - Utilizar distância euclidiana

Dado um exemplo de teste x_q :

1. Calcule a distância entre x_q e todas as amostras de treinamento.
2. Encontre os k vizinhos mais próximos.
3. **Classificação:** atribua a classe mais frequente entre os k vizinhos.

- Aplicar validação cruzada com 3 folds para encontrar o melhor valor de K

Exercício (Extra - 2)



- Escreva as letras A e I no papel
- Tire uma foto com seu celular
- Carregue a imagem para seu computador
- Aplique as transformações necessárias na imagem (tamanho, imagem na escala cinza, ajuste de brilho, etc)
- Utilize o algoritmo treinado para classificar a imagem